Full-Stack Engineering

Project Report

Semester-V (Batch-2023)

'SwiftSend' - (File Sharing System)

Supervised By:

Md. Nurul Islam
FSE Trainer

Submitted By:

Kunal Bhatia, 23109906038(G8)
Lakshay Garg, 2310990640(G8)
Mudit Kalra, 2310990660(G8)
Priyanshu, 2310991183(G8)

Department of Computer Science and Engineering
Chitkara University Institute of Engineering & Technology,
Chitkara University, Punjab

ABSTRACT

The File Sharing Platform is a full-stack web application designed to provide a secure, organized, and efficient way for users to store, manage, and share digital files. In today's digital world, individuals and organizations frequently need to transfer files, collaborate on data, and maintain structured storage systems. However, many existing solutions are either too complex, lack transparency, or are not suitable for learning and customization. This project addresses these gaps by providing a simplified yet functional file-sharing system built using modern web technologies.

The platform incorporates secure authentication using JSON Web Tokens (JWT), ensuring that only authorized users can access protected resources. Passwords are encrypted using bcrypt, which prevents unauthorized access even if database data is compromised. Once logged in, users can upload files, create folders, and organize them in a nested structure similar to traditional file systems. This hierarchical organization improves usability and allows users to manage large amounts of data efficiently.

One of the most important features of the system is its sharing capability. Users can share files and folders with other registered users, enabling collaboration within the platform. Additionally, the system allows the generation of public links, making it possible to share files with external users who do not have accounts. This flexibility ensures both controlled access and ease of sharing. Users can also revoke access at any time, maintaining full control over their data.

The platform supports both local storage and optional cloud storage integration, making it adaptable for different deployment environments. Features like breadcrumb navigation and dashboard views enhance the user experience by providing intuitive navigation and organization. Overall, the project demonstrates practical implementation of authentication, file handling, database management, and access control, making it a strong foundation for understanding real-world file-sharing systems.

INDEX

1. INTRODUCTION

1.1 OVERVIEW OF THE PROJECT

1.2 PROJECT GOALS

1.3 MOTIVATION BEHIND THE PROJECT

1.4 EDUCATIONAL AND PRACTICAL BENEFITS

2. **PROBLEM DEFINITION AND REQUIREMENTS**

2.1 INTRODUCTION TO PROBLEM

2.2 CHALLENGES ADDRESSED BY THE SYSTEM

2.3 FUNCTIONAL REQUIREMENTS

2.4 NON-FUNCTIONAL REQUIREMENTS

3. **PROPOSED DESIGN AND METHODOLOGY**

3.1 DATA STORAGE AND MANAGEMENT

3.2 VERSION CONTROL SYSTEM

3.3 BACKUP STRATEGY AND SCHEDULING

3.4 FILE CHANGE AND DETECTION TRACKING

3.5 SECURITY AND DATA INTEGRITY

4. PROPOSED RESULT

5. REFERENCES

1. INTRODUCTION

1.1 Overview of the Project

The File Sharing Platform is a web-based system that allows users to securely upload, manage, and share files. It provides a centralized environment where users can store their data and access it anytime through authentication.

The system is designed with both backend and frontend components. The backend handles operations like authentication, file storage, and database management, while the frontend provides an interface for user interaction. Users can create folders and organize files in a hierarchical structure, which makes file management more intuitive.

Additionally, the platform includes sharing features that allow users to collaborate. Files and folders can be shared with specific users or through public links. This ensures flexibility in how data is accessed and distributed.

1.2 Project Goals

The main goal of this project is to develop a secure and efficient file-sharing system that meets user needs for storage and collaboration.

- To allow users to upload and organize files in folders
- To provide secure authentication using JWT
- To enable controlled sharing between users
- To support public file sharing through links
- To ensure data security through encryption techniques
- To create a user-friendly interface for easy navigation

These goals ensure that the platform is both functional and practical for real-world usage.

1.3 Motivation Behind the Project

The motivation for this project comes from the growing need for digital file management systems. Users often struggle with organizing files and sharing them securely across platforms.

Many existing solutions are either too complex or not suitable for learning purposes. This project aims to provide a simplified system that demonstrates how file-sharing platforms work internally. It also helps developers understand important concepts like authentication, file handling, and database management.

Another motivation is to gain hands-on experience in full-stack development and build a project that can be used in real-world scenarios.

1.4 Educational and Practical Benefits

This project provides several learning and practical advantages:

- It helps understand how authentication systems like JWT work
- It demonstrates secure password handling using bcrypt
- It provides experience with file uploads and storage systems
- It teaches how to design and manage databases
- It improves skills in API development and backend logic

From a practical perspective, it prepares developers for building real-world applications and enhances their problem-solving abilities.

2. PROBLEM DEFINITION AND REQUIREMENTS

2.1 Introduction to Problem

Managing files securely and efficiently is a major challenge in today's digital environment. Users need systems that allow them to store, organize, and share files without risking data loss or unauthorized access.

Traditional methods like email or basic storage lack proper organization and security. This creates the need for a system that combines structured storage with secure sharing capabilities.

2.2 Challenges Addressed by the System

The system solves multiple real-world challenges:

- Ensures secure access using authentication mechanisms
- Provides structured file organization using folders
- Allows controlled sharing between users
- Enables public sharing without compromising ownership
- Maintains data integrity and consistency

These challenges are critical for building a reliable file-sharing system.

2.3 Functional Requirements

The platform includes the following functionalities:

- User registration and login system
- Secure authentication using JWT tokens
- Password encryption using bcrypt

- File upload with size limits
- Folder creation and nested directory structure
- Rename and delete functionality for files/folders
- File download capability
- Dashboard for viewing personal files
- “Shared with Me” section for received files
- Breadcrumb navigation for easy movement
- User-to-user sharing with read-only access
- Public link generation for external sharing
- Access revocation for shared files
- User search functionality
- Support for local and cloud storage

2.4 Non-Functional Requirements

The system must satisfy the following:

- Security: Protect user data through authentication and encryption
- Performance: Handle file uploads and downloads efficiently
- Scalability: Support increasing users and storage
- Usability: Provide an intuitive interface
- Reliability: Ensure consistent system availability

3. PROPOSED DESIGN AND METHODOLOGY

3.1 Data Storage and Management

The system stores file metadata in a database, including file name, size, type, and owner details. This helps in efficient retrieval and management of files.

Actual files are stored either locally or in cloud storage. Folder structures are maintained using hierarchical relationships, allowing nested directories. This ensures organized storage and easy navigation.

3.2 Version Control System

Version control is used to manage the development process of the project. Tools like Git help track changes, maintain code history, and support collaboration.

It allows developers to revert to previous versions if errors occur and ensures smooth project development.

3.3 Backup Strategy and Scheduling

The system ensures data availability by storing files securely. Local storage provides immediate access, while cloud storage adds reliability.

Although advanced scheduling may not be implemented, the storage system ensures that files are preserved and accessible during normal operation.

3.4 File Change and Detection Tracking

The platform tracks file operations such as uploads, renaming, and deletion. These changes are updated in real-time in both the database and user interface.

This ensures that users always see the latest version of their data and prevents inconsistencies.

3.5 Security and Data Integrity

Security is maintained through multiple mechanisms:

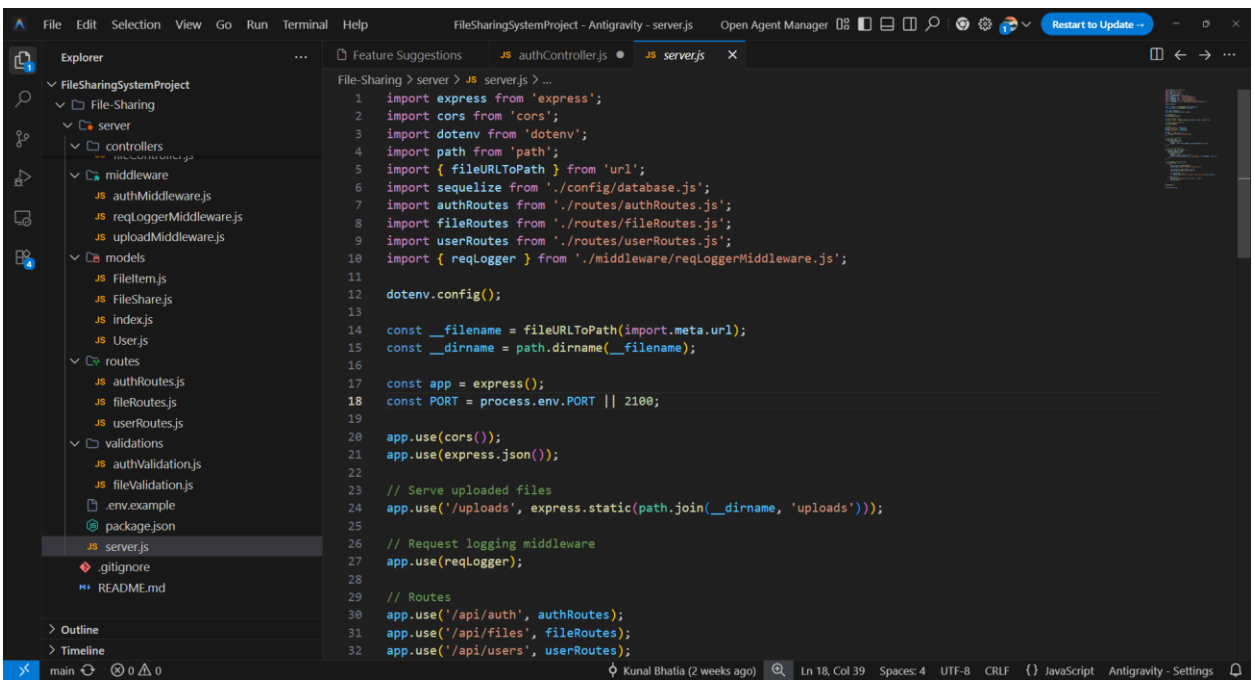
- JWT authentication secures API endpoints
- bcrypt protects user passwords
- Access control ensures only authorized users can access files
- Public links allow controlled external access
- Users can revoke permissions to maintain control

4. PROPOSED RESULT

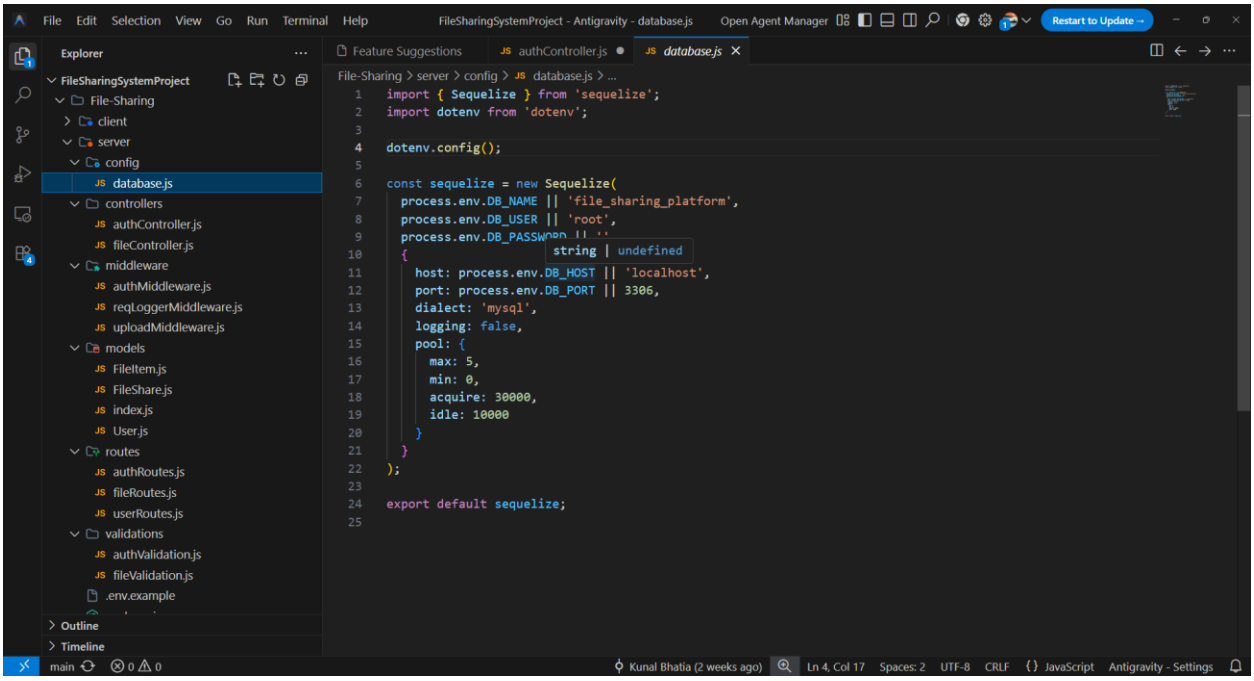
The system successfully provides a secure platform for file management and sharing. Users can upload files, organize them, and share them with others efficiently.

The platform demonstrates the integration of authentication, storage, and sharing features into a single system. It meets the requirements of usability, security, and functionality, making it suitable for both learning and basic real-world applications.

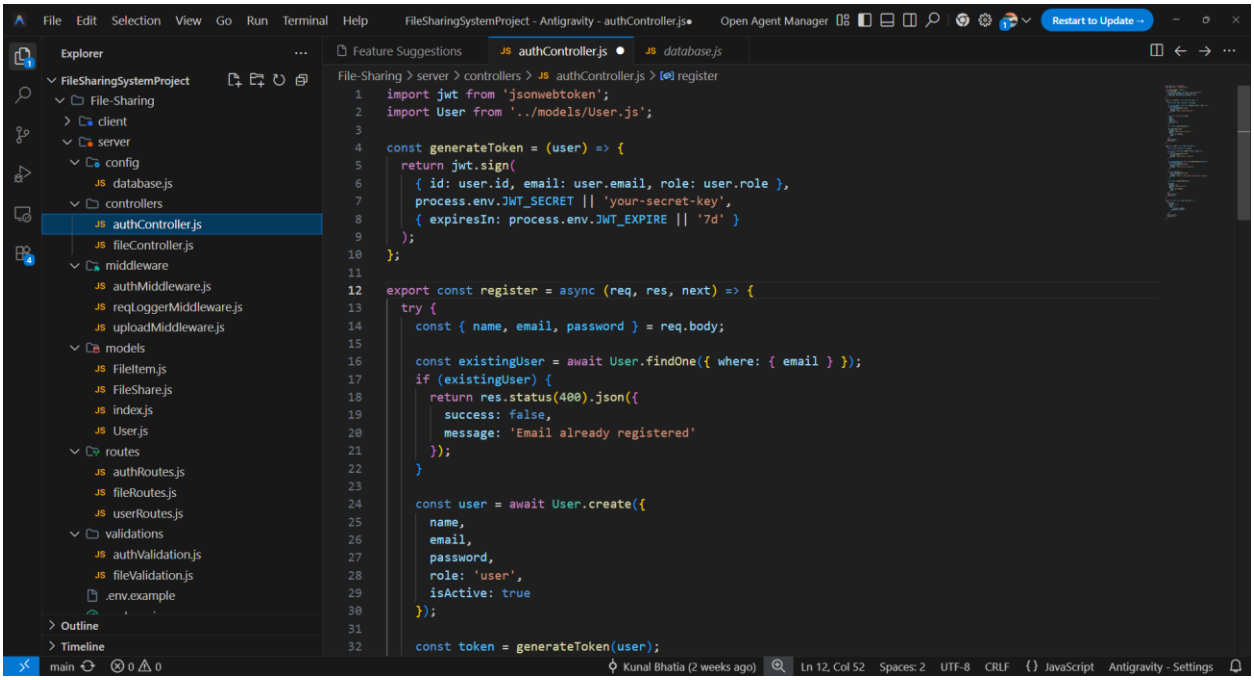
1. Main Server.js File



2. database.js Config File



3. authController.js Controller File



4. authMiddleware.js Middleware File

File Edit Selection View Go Run Terminal Help FileSharingSystemProject - Antigravity - authMiddleware.js Open Agent Manager Restart to Update

Explorer

FileSharingSystemProject
- File-Sharing
  - client
  - server
    - config
      - database.js
    - controllers
      - authController.js
      - fileController.js
    - middleware
      - authMiddleware.js
      - reqLoggerMiddleware.js
      - uploadMiddleware.js
    - models
      - FileItem.js
      - FileShare.js
      - index.js
      - User.js
    - routes
      - authRoutes.js
      - fileRoutes.js
      - userRoutes.js
    - validations
      - authValidation.js
      - fileValidation.js
    - .env.example

Outline
Timeline

Feature Suggestions | authController.js | authMiddleware.js

File-Sharing > server > middleware > authMiddleware.js > ...

```
import jwt from 'jsonwebtoken';
import User from '../models/User.js';

export const authenticate = async (req, res, next) => {
  try {
    const authHeader = req.headers.authorization;

    if (!authHeader || !authHeader.startsWith('Bearer ')) {
      return res.status(401).json({
        success: false,
        message: 'Access denied. No token provided.'
      });
    }

    const token = authHeader.split(' ')[1];

    const decoded = jwt.verify(token, process.env.JWT_SECRET || 'your-secret-key');

    const user = await User.findByPk(decoded.id);

    if (!user) {
      return res.status(401).json({
        success: false,
        message: 'Invalid token. User not found.'
      });
    }

    if (!user.isActive) {
      return res.status(403).json({
        success: false,
        message: 'Account is deactivated.'
      });
```

main 0 0 Ln 1, Col 1 Spaces: 2 UTF-8 CRLF {} JavaScript Antigravity - Settings

5. User.js Modals File

File Edit Selection View Go Run Terminal Help FileSharingSystemProject - Antigravity - User.js Open Agent Manager Restart to Update

Explorer

FileSharingSystemProject
- File-Sharing
  - client
  - server
    - config
      - database.js
    - controllers
      - authController.js
      - fileController.js
    - middleware
      - authMiddleware.js
      - reqLoggerMiddleware.js
      - uploadMiddleware.js
    - models
      - FileItem.js
      - FileShare.js
      - index.js
      - User.js
    - routes
      - authRoutes.js
      - fileRoutes.js
      - userRoutes.js
    - validations
      - authValidation.js
      - fileValidation.js
    - .env.example

Outline
Timeline

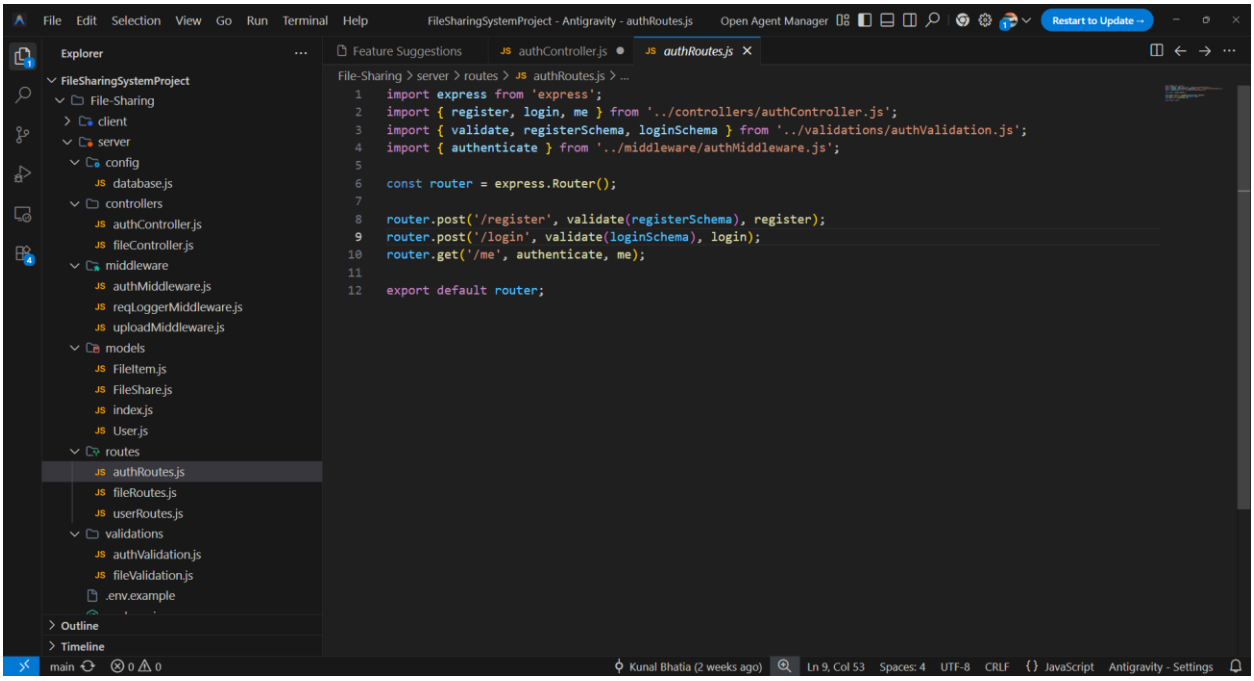
Feature Suggestions | authController.js | User.js

File-Sharing > server > models > User.js > ...

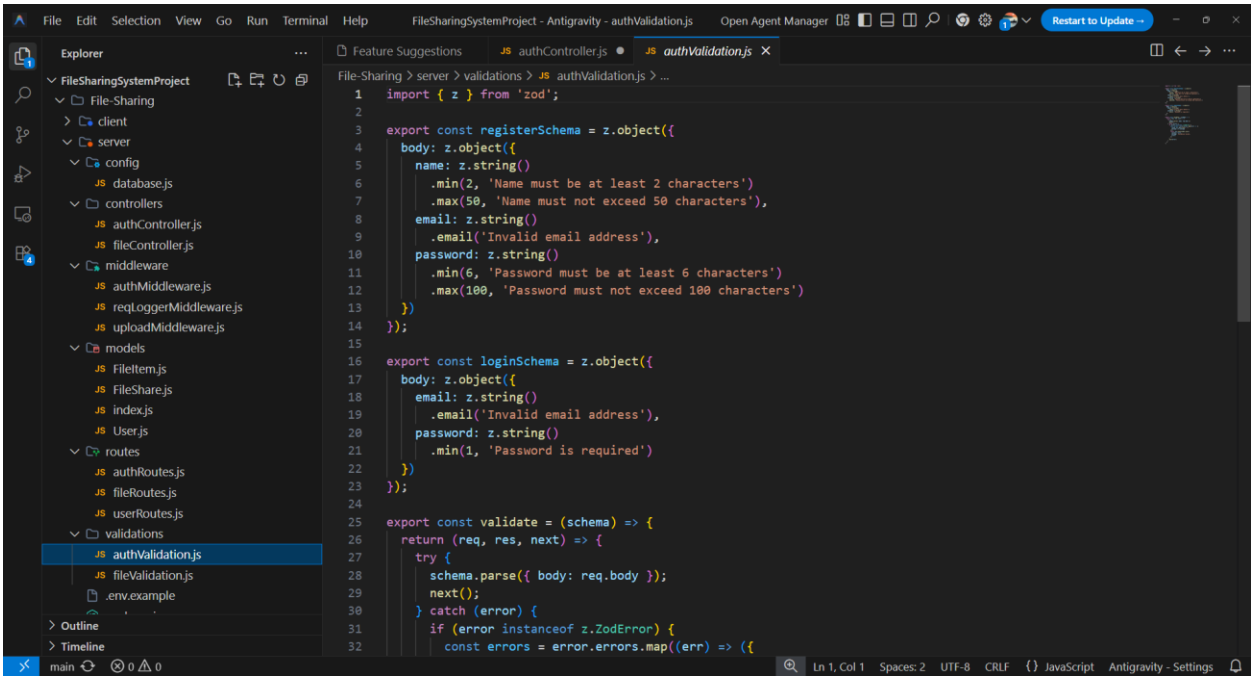
```
import { DataTypes } from 'sequelize';
import sequelize from '../config/database.js';
import bcrypt from 'bcryptjs';

const User = sequelize.define('User', {
  id: {
    type: DataTypes.INTEGER,
    primaryKey: true,
    autoIncrement: true
  },
  name: {
    type: DataTypes.STRING,
    allowNull: false,
    validate: {
      notEmpty: true
    }
  },
  email: {
    type: DataTypes.STRING,
    allowNull: false,
    unique: true,
    validate: {
      isEmail: true,
      notEmpty: true
    }
  },
  password: {
    type: DataTypes.STRING,
    allowNull: false,
    validate: {
      notEmpty: true
    }
```

main 0 0 Ln 1, Col 1 Spaces: 2 UTF-8 CRLF {} JavaScript Antigravity - Settings

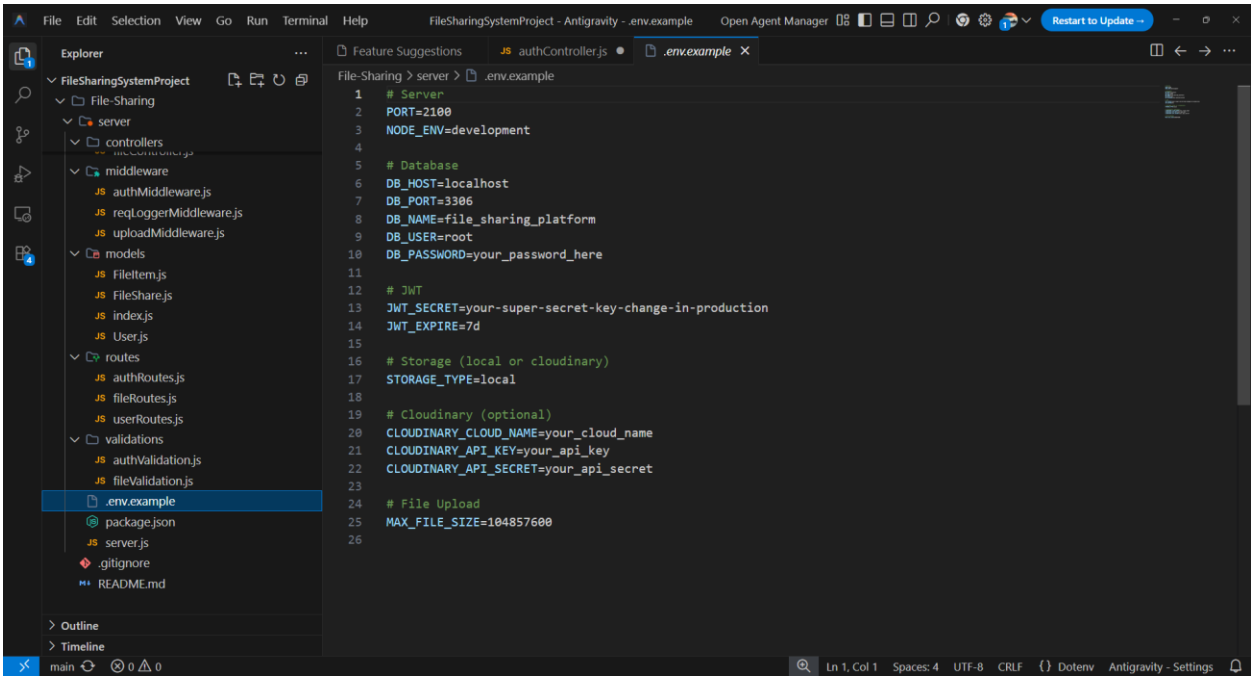
6. authRoutes.js Router File



7. authValidation.js Validation File



8. env.example File For APIs



5. REFERENCES

- JWT Documentation

  https://www.jwt.io/introduction#what-is-json-web-token

- bcrypt Documentation

  https://www.npmjs.com/package/bcrypt

- Cloudinary Documentation

  https://cloudinary.com/documentation

- Node.js Documentation

  https://nodejs.org/docs/latest/api/

- Express.js Documentation

  https://expressjs.com/

- MongoDB Documentation

  https://www.mongodb.com/docs/

- REST API Design Principles

  https://restfulapi.net/